ECE 257B Final Report

Andrew Fu, Alexis Emmanuelle Pascual, Yusuf Amanullah

March 2026

Abstract

Wireless communication systems increasingly rely on machine learning techniques operating directly at the physical layer to perform tasks such as signal detection, protocol identification, device fingerprinting, and anomaly detection. These systems must operate in complex, interference-rich spectrum environments characterized by overlapping transmissions, hardware impairments, and non-ideal noise characteristics. However, most existing research is evaluated either on synthetic simulations with perfect labels or on real-world spectrum measurements with incomplete or unreliable ground truth. Extracting ground truth from real-world spectrum is problematic due to overlapping time and frequency domain transmissions, clock drift, fading and interference effects, and various hardware distortions.

In this project, we focus on implementing a replay-based labeling framework that combines software-defined radios with protocol-aware sniffers to precisely control, inject, and recover labeled wireless transmissions in realistic RF environments, and a controlled signal-injection pipeline that embeds fully labeled signals into real background recordings while preserving the receiver's true noise characteristics. However, using only single-receiver (SISO) observations may limit the reliability of ground-truth generation. We further propose a system leverages single-input multiple-output (SIMO) reception via a multi-antenna USRP to observe the same narrowband BLE channel, creating a more complete and robust dataset.

1 Project Introduction

The project aims at developing a high-fidelity, real-world RF dataset and benchmarking framework for evaluating machine learning algorithms on low-power IoT wireless protocols, with a focus on BLE. The realism of the dataset comes from capturing signals directly from the wireless spectrum using software-defined radios (USRPs) and protocol-aware BLE sniffers, rather than relying solely on simulation. The project is proposed to involve four major steps:

- **Capturing real-world BLE signals using USRP and sniffer:** We plan to use a multi-channel USRP with two receive antennas to capture RF signals, while a protocol-aware BLE sniffer provides semantic ground truth (device ID, packet type, timing, etc.) for alignment and labeling.
- **Signal processing and packet recovery using the sniffer:** Primary packet decoding will be performed using the hardware BLE sniffer, which provides reliable demodulation, decoding, and metadata extraction for BLE transmissions. The USRP-based IQ recordings will be used for additional signal analysis, including timing alignment, phase correction, and maximum-SNR antenna selection.
- **Time Alignment:** Now that the sniffer metadata and USRP IQ streams are synchronized in the previous step, we can now do precise timestamp conversion and packet matching, generating accurately labeled, time-aligned datasets.
- **Training and evaluation of ML models in Python:** The time-aligned, fully labeled IQ datasets will be used in Python to train and evaluate machine learning models for tasks such as protocol identification, device recognition, energy detection, and anomaly detection. These experiments will quantify the performance gap between real-world wideband RF environments and idealized simulation-based benchmarks.

1.1 Background & Design Overview

Prior work in RF machine learning has demonstrated strong performance on tasks such as modulation recognition, protocol classification, and device identification using curated over-the-air measurements [1]. These methods have been extended toward various IoT applications, with many works citing the need for improved security of IoT devices [2] [3].

However, single-antenna BLE capture experiments often struggle with low-SNR packets, provide limited spatial information, and are sensitive to multipath fading. Such setups can reliably log only strong packets, and RSSI-based inference suffers due to a lack of spatial diversity.

MIMO and multi-antenna systems have demonstrated significant improvements in observability of weak or overlapping signals, more accurate timing and clock alignment, and the ability to exploit spatial diversity. Inspired by this, we hypothesize that a SIMO system with appropriate antenna spacing can provide multiple spatial observations of the same transmission. Maximum-SNR antenna selection allows us to choose the antenna branch that provides the strongest and most reliable observation for each packet. This approach improves recoverability of weak or fading signals while maintaining robustness in the presence of multipath and interference, leading to more consistent alignment between sniffer-reported metadata and raw IQ observations.

By applying machine learning algorithms to this spatially enhanced dataset, we can reliably identify protocols and devices. This enables practical applications such as securing smart homes and IoT environments by detecting unauthorized devices, preventing rogue access, and ensuring reliable operation of everyday connected appliances.

1.2 USRP SIMO Setup

In our experiment, we capture BLE packets using a two-antenna setup with USRP receivers. Each antenna provides a separate stream of samples, with the antennas positioned approximately 5 meters apart. This spatial separation is critical for our application, as we aim to perform RSSI-based angle estimation. The greater the distance between the antennas, the more pronounced the differences in received signal strength, improving the system's ability to resolve the direction-of-arrival of BLE packets.

Each capture stream has its own start time: for instance, the second antenna's stream (s_2) begins approximately 30 ms after the first antenna's stream (s_1). The precise start timestamps for each stream are recorded in the associated SigMF metadata files, allowing for accurate time alignment between the two streams. After alignment, the two signals can be treated as a SIMO (Single Input, Multiple Output) system, which enhances our ability to detect and recover packets, particularly those with weak received power that might otherwise be lost in a single-antenna capture.

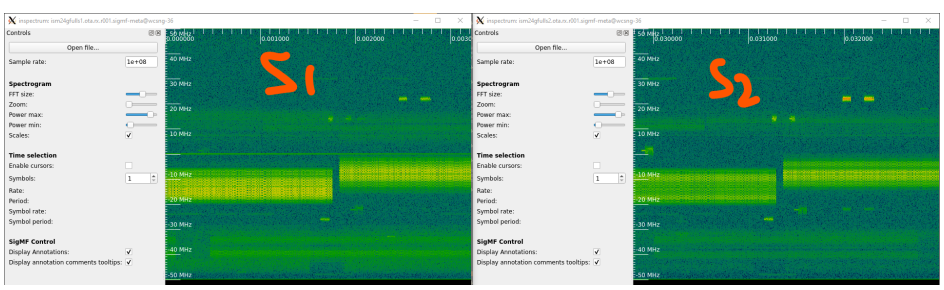


Figure 1: Spectrum of Antenna 1 and 2

1.3 Advantages over Single-Antenna Captures

Single-antenna BLE captures suffer from several limitations:

- **Packet loss due to weak signals:** Packets transmitted at the edge of the coverage area may be too weak to detect reliably.

- **Limited SNR diversity:** With only one measurement of signal strength, it is difficult to distinguish between low-power packets and noise, reducing the reliability of RSSI-based inference.
- **No spatial diversity:** A single antenna cannot leverage differences in received power across space, which limits the accuracy of angle estimation and SIMO processing.

By contrast, our two-antenna SIMO setup offers the following improvements:

- **Enhanced packet recovery:** Using both streams allows us to select the antenna with the highest SNR for each packet, recovering weak packets that might be missed with a single antenna.
- **Spatial diversity for RSSI-based estimation:** The 5 m separation increases the observed SNR differences, providing better resolution for angle-of-arrival calculations.
- **Robustness to channel fading and interference:** Even if one antenna receives a packet in a deep fade, the other may still capture it clearly, improving the overall reliability of the ground truth table.

2 Technical Approach

2.1 Sniffer Pipeline (Protocol Layer)

RF capture (BLE) → Sniffer → Metadata extraction (Device ID, channel index, packet type, payload length)

This will be handled by the sniffer, although we considered implementing it using GNU Radio as a fallback, if the sniffer was not successful in decoding all packets.

2.2 Fusion Pipeline (Time-Alignment & Labeling Layer)

Sniffer Metadata + USRP IQ data → Clock Synchronization → Protocol Packet and IQ burst matching → Labeled packet objects → Dataset generation

2.3 Clock Synchronization

Clock synchronization aligns the USRP sample timeline with the BLE sniffer timestamp domain, enabling RF IQ bursts to be matched to protocol-layer packets. Because the USRP and sniffer operate on independent clocks, both offset and drift must be estimated. We implemented two methods of clock synchronization: global clock drift and offset estimation for fine-grained correction, and incremental piecewise time correction for coarse-grained correction.

2.3.1 Global Clock Drift and Offset Estimation

Since sync packets are transmitted periodically with interval T_s, the ideal timeline is constructed as:

$$\tau_i = n_i T_s$$

where:

$$n_i = \sum_{j=2}^{i} \text{round}\left(\frac{s_j - s_{j-1}}{T_s}\right), \quad n_0 = 0 \tag{1}$$

The relationship between observed and ideal time is modeled as:

$$s_i = \alpha \tau_i + \beta \tag{2}$$

The parameters α and β are estimated via least squares:

$$(\hat{\alpha}, \hat{\beta}) = \arg\min_{\alpha,\beta} \sum_{i=1}^{M} (s_i - (\alpha\tau_i + \beta))^2 \tag{3}$$

This corresponds to the closed-form solution:

$$\hat{\alpha} = \frac{\sum_i (\tau_i - \bar{\tau})(s_i - \bar{s}_i)}{\sum_i (\tau_i - \bar{\tau})^2}, \quad \hat{\beta} = \bar{s} - \hat{\alpha}\bar{\tau} \tag{4}$$

The clock drift relative to the nominal interval is: $\delta = \hat{\alpha} - T_s$, and clock offset is $\hat{\beta}$.

2.3.2 Incremental Piecewise Time Correction

Define the observed inter-sync intervals:

$$\Delta s_i = s_i - s_{i-1} \tag{5}$$

The corresponding ideal intervals are:

$$\Delta \tilde{s}_i = \text{round}\left(\frac{\Delta s_i}{T_s}\right) T_s \tag{6}$$

The timing error for each interval is:

$$e_i = \Delta s_i - \Delta \tilde{s}_i \tag{7}$$

The accumulated timing error up to sync index i is:

$$E_i = \sum_{j=2}^{i} e_j, \quad E_1 = 0 \tag{8}$$

For any packet timestamp t_k occuring between packets s_i and s_{i+1}, the corrected timestamp is:

$$t_k^{\text{corr}} = t_k - E_i \tag{9}$$

This results in a piecewise-constant correction function:

$$t_k^{\text{corr}} = t_k - E(t_k) \tag{10}$$

where $E(t_k) = E_i$ for $t_k \in [s_i, s_{i+1})$.

2.4 USRP Pipeline (Signal Layer) (SIMO Case)

USRP IQ capture (dual antenna) → Antenna Time Alignment → Sniffer-Based Packet Extraction → Per-Packet DSP → Antenna Selection

SISO Case The implementation of the SISO processing pipeline closely mirrors that of the SIMO case, with the primary difference being the absence of antenna selection. In the SISO setting, packet extraction is performed using a single stream of IQ samples, whereas the SIMO framework extends this by incorporating multiple antenna observations and selecting the highest-SNR packet on a per-instance basis. Aside from this selection step, the remaining processing stages—including channel isolation, timestamp-based packet extraction, and post-processing—remain largely identical.

2.4.1 Packet Detection and Device Separation

Packet detection and device separation are performed using synchronized BLE sniffer data rather than signal-domain detection. The sniffer provides precise packet-level metadata, including timestamps, channel index, payload length, and device identifiers (e.g., access address), which are used to directly extract packets from the USRP IQ data.

- **Antenna Time Alignment:**

The two USRP receive streams are not hardware-synchronized and exhibit a fixed time offset (on the order of milliseconds). Let t_1 and t_2 denote the start times of antenna 1 and antenna 2 captures, respectively, obtained from metadata. The relative offset is

$$\Delta t = t_2 - t_1, \quad \Delta n = \lfloor \Delta t \cdot F_s \rfloor, \tag{11}$$

where F_s is the sampling rate. One stream is shifted by Δn samples to align both antennas to a common time reference.

- **Sniffer-Based Packet Extraction:**

Let t_k denote the timestamp of packet k from the BLE sniffer. After synchronization between the sniffer and USRP time axes, packet start indices are computed as

$$n_k = \lfloor t_k F_s \rfloor. \tag{12}$$

Packet lengths are obtained directly from the sniffer metadata, which provides the exact number of bytes for each packet. Let B_k denote the total packet length (in bytes) reported by the sniffer. The corresponding sample length is

$$L_k = \left\lfloor B_k \cdot \frac{8F_s}{R_b} \right\rfloor, \tag{13}$$

where $R_b = 1$ Mbps is the BLE data rate.

Packets are then extracted directly from the aligned USRP streams:

$$r_i^{(k)}[n] = r_i[n_k + n], \quad n = 0, \ldots, L_k - 1. \tag{14}$$

This approach preserves variable packet durations and avoids truncation or inclusion of noise that can occur when assuming a fixed packet length. It also eliminates false detections and missed packets inherent to energy-based or correlation-based methods.

- **Device Separation via Sniffer Labels:**

Device separation is performed using sniffer-provided identifiers such as access address and channel index. Packets are grouped according to these labels, producing device-specific IQ streams. This avoids reliance on spatial clustering or angle-of-arrival (AoA) estimation.

Note on AoA Estimation: Although two spatially separated antennas are available, phase-based AoA estimation is not used. The receive chains are not phase-synchronized, introducing unknown and time-varying phase offsets. Furthermore, the large antenna separation (5 m) leads to phase wrapping at 2.4 GHz, making the phase difference ambiguous. Without precise calibration, this prevents reliable AoA estimation. Therefore, spatial processing is limited to SNR-based selection and combining.

2.4.2 Per-Packet Processing

All signal processing is performed independently on each extracted packet.

- **Maximum-SNR Antenna Selection:** For each detected packet, the signal-to-noise ratio (SNR) is estimated independently for each antenna using the raw extracted IQ samples:

$$\mathrm{SNR}_i^{(k)} = \frac{\mathbb{E}[|r_i^{(k)}[n]|^2]}{\mathrm{Var}(r_i^{(k)}[n])}, \tag{15}$$

  where $r_i^{(k)}[n]$ denotes the unprocessed packet samples from antenna i.

  The antenna with the highest SNR is selected:

$$i^\star = \arg\max_i \mathrm{SNR}_i^{(k)}. \tag{16}$$

Only the selected packet $r_{i^\star}^{(k)}[n]$ is retained for subsequent processing, ensuring that all downstream operations (e.g., DC offset removal, normalization, and CFO correction) are applied to the highest-quality observation.

- **DC Offset Removal:** Each packet is mean-subtracted:

$$\tilde{r}_{i^\star}^{(k)}[n] = r_{i^\star}^{(k)}[n] - \frac{1}{L_k}\sum_{m=0}^{L_k-1} r_{i^\star}^{(k)}[m]. \tag{17}$$

- **Amplitude Normalization:** Ensures consistent power scaling across antennas and packets:

$$\hat{r}_{i^\star}[n] = \frac{\tilde{r}_{i^\star}[n]}{\sqrt{\mathbb{E}[|\tilde{r}_{i^\star}[n]|^2]}}. \tag{18}$$

- **CFO Estimation and Correction:** Carrier frequency offset is estimated using phase differences over the packet preamble:

$$\Delta\phi[n] = \arg\left(\hat{r}_v^{(k)}[n]\hat{r}_{i^\star}^{(k)*}[n-1]\right), \tag{19}$$

$$\hat{f}_{\text{CFO}} = \frac{F_s}{2\pi}\cdot\frac{1}{M}\sum_{n=1}^{M}\Delta\phi[n], \tag{20}$$

where M is the number of samples used (e.g., preamble length). CFO is corrected as

$$r_{i^\star,\text{corr}}^{(k)}[n] = \hat{r}_{i^\star}^{(k)}[n]\exp\left(-j2\pi\hat{f}_{\text{CFO}}\frac{n}{F_s}\right). \tag{21}$$

- **Phase Tracking:** Residual phase drift is removed by fitting and subtracting a linear phase trend:

$$\phi[n] = \arg(r_{i^\star,\text{corr}}^{(k)}[n]), \quad \tilde{\phi}[n] = \text{unwrap}(\phi[n]), \tag{22}$$

$$\phi_{\text{track}}[n] = \tilde{\phi}[n] - (an+b), \tag{23}$$

where a and b are obtained via least-squares fitting.

3 Results

Our key contributions so far are as follows:

3.1 BLE Sniffer

Configuration of the BLE sniffer posed a major challenge for completing our hardware setup for this project. We used Nordic's nRF52840 DK as our BLE sniffer. The board is low-cost and has the advantage of many well-documented tools for use as a BLE sniffer. However, the board suffers from the hardware limitation of being unable to monitor all BLE channels at once in a wideband manner; because of this limitation, the official nRF sniffing firmware is unable to sniff existing connections between 2 BLE devices without first observing the CONNECT_IND packet that defines the channel hop sequence for that connection. With the standard firmware, if the initial connection between 2 devices is detected, we can still sniff data packets in the connection. The following image shows a Wireshark capture of empty data packets (in BLE, used to maintain a connection between 2 devices if there is no data sent otherwise) in a dummy connection between an iPhone and iPad, established using Nordic's nRF connect app.

3.1.1 Alternative Firmware Options for nrf52840

We attempted to overcome this hurdle by extensively exploring an alternative firmware for the nRF52840, called ButteRFly, which is built for the WHAD platform for wireless hacking. Unlike the default Nordic

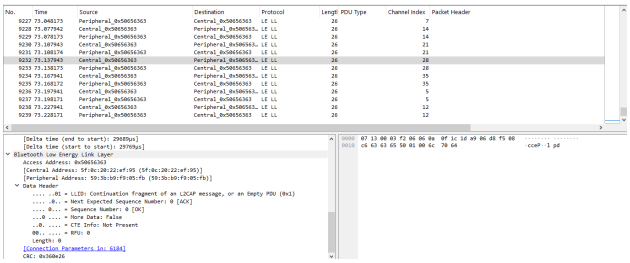


Figure 2: PCAP with dummy BLE connection between 2 devices

firmware, ButteRFly can intercept existing BLE connections by scanning data channels, attempting to identify the BLE access address by correlating to identify a repeating 32-bit pattern, and reverse engineering the hop pattern. The WHAD platform provides command line tools for sniffing packets and recording, much like Nordic's extcap integration with Wireshark for BLE sniffing.

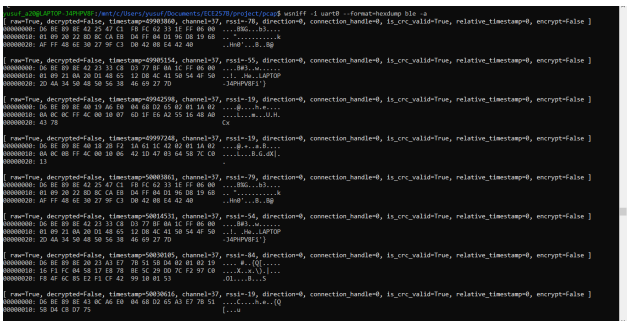


Figure 3: Sniffing BLE advertisements with WHAD ButteRFly

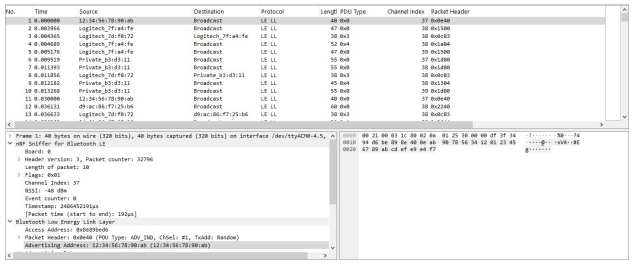
While ButteRFly looked promising as the right tool for our unlocking our sniffer's capabilities, as an open source project, the firmware suffers from limited available documentation and a relatively small user base making it difficult to receive support for technical issues. In particular, a major limitation of ButteRFly is that certain boards do not have an explicitly compatible version, including our nRF52840-DK board. We tried to use a version of the firmware targeting the nRF52840 USB dongle- a different board- which was mostly functional, but some functions were at risk of not working due to mismatch in I/O pin assignment/definitions. We attempted to circumvent potential issues by recompiling the ButteRFly firmware to support our nRF52840-DK board, yet we still struggled with getting the firmware to track existing connections. Our tests with firmware did prove successful in sniffing access addresses from existing BLE connections — indicating that existing BLE connections were visible to the new firmware, yet intercepting actual data packets could not be achieved.

With the technical challenges posed by the sniffer firmware too big to overcome despite our significant time investment up to that point, we pivoted back to using the stock Nordic sniffing firmware for the nf52840, and narrowed the scope of our research to only advertising BLE packets.

3.1.2 Time Synchronization

We tackled the problem of clock synchronization by creating our own dummy BLE packets and injecting them into our stored replay at known timestamps at a fixed interval. We are able to inject sync packets purely using software-based post-processing of the replay signal by simply adding the IQ samples from our created BLE waveform to the replay samples, eliminating the possibility of any additional hardware-based timing offset from streaming the packets over the air. The dummy BLE packet must be an advertisement packet, keeping consistent with the sniffer's ability to view only advertisement packets at the start of a capture. Our created BLE packet had a minimal length of 40 bytes and sat on channel 37, and was repeated ever 10 ms over the 1 second period of the capture, starting from sample 0. Upon replay of the new output

SigMF file, the packet was detected in Wireshark as shown in the figure below:



No.	Time	Source	Destination	Protocol	Length	PDU Type	Channel Index	Packet Header
1	0.000000	12:34:56:78:90:ab	Broadcast	LE LL	40	0x0	37	0x0e40
2	0.003966	Logitech_7f:a4:fe	Broadcast	LE LL	47	0x0	38	0x1540
3	0.004365	Logitech_7d:f0:72	Logitech_7f:a4:fe	LE LL	38	0x3	38	0x0c03
4	0.004689	Logitech_7f:a4:fe	Broadcast	LE LL	52	0x4	38	0x1a04
5	0.005176	Logitech_7f:a4:fe	Broadcast	LE LL	47	0x0	39	0x1500
6	0.009519	Private_b3:d3:11	Broadcast	LE LL	55	0x0	37	0x1d00
7	0.011393	Private_b3:d3:11	Broadcast	LE LL	55	0x0	38	0x1d00
8	0.011856	Logitech_7d:f0:72	Private_b3:d3:11	LE LL	38	0x3	38	0x0c03
9	0.012182	Private_b3:d3:11	Broadcast	LE LL	45	0x4	38	0x1304
10	0.013268	Private_b3:d3:11	Broadcast	LE LL	55	0x0	39	0x1d00
11	0.030000	12:34:56:78:90:ab	Broadcast	LE LL	40	0x0	37	0x0e40
12	0.036131	d9:ac:86:f7:25:b6	Broadcast	LE LL	60	0x0	38	0x2240
13	0.036633	Logitech_7d:f0:72	d9:ac:86:f7:25:b6	LE LL	38	0x3	38	0x0c03

```
> Frame 1: 40 bytes on wire (320 bits), 40 bytes captured (320 bits) on interface /dev/ttyACM0-4.5,
v nRF Sniffer for Bluetooth LE
    Board: 0
  > Header Version: 3, Packet counter: 32796
    Length of packet: 18
  > Flags: 0x01
    Channel Index: 37
    RSSI: -46 dBm
    Event counter: 0
    Timestamp: 2460452191µs
    [Packet time (start to end): 192µs]
v Bluetooth Low Energy Link Layer
    Access Address: 0x8e89ed6
  > Packet Header: 0x0e40 (PDU Type: ADV_IND, ChSel: #1, TxAdd: Random)
    Advertising Address: 12:34:56:78:90:ab (12:34:56:78:90:ab)

0000  00 21 00 03 1c 00 02 0a  01 25 30 00 00 df 3f 34   ·!······ ·%0···?4
0010  94 d6 be 89 0e 40 0e ab  90 78 56 34 12 01 23 45   ·····@·· ·xV4··#E
0020  67 89 ab cd ef e9 e4 f7                            g·******
```

Figure 4: Synthetic BLE Advertisement Detected by Sniffer

3.2 USRP (Andrew + Alexis)

Our project uses the Ettus Research USRP X300 to perform both IQ capture and sync injection events. IQ captures were made from the ambient RF data in lab room 1601 in Atkinson Hall. The captured data predominantly contains 802.11, BLE, and ZigBee.

Captures were made centered at 2.45 GHz and at a sampling rate of 1 MHz. Transmissions were made with the same center frequency and sampling rate, and streamed with a buffer size of 1048576.

3.3 Time Synchronization Results

We first evaluate packet extraction performance in the single-antenna (SISO) setting using sniffer-derived timestamps and packet length information. Packet start indices are computed directly from the sniffer timestamps, and BLE packets are extracted from the corresponding IQ samples.

```
Clock drift (per second) and offset:
1.090908e-06 1.645987e-04

         0         0
    0.0300    0.0300
    0.0900    0.0900
    0.1102    0.1100
    0.1502    0.1500
    0.1802    0.1800
    0.1902    0.1900
    0.2102    0.2100
    0.2402    0.2400
    0.2502    0.2500
    0.2802    0.2800
    0.2902    0.2900
    0.3002    0.3000
    0.3302    0.3300
    0.3402    0.3400
    0.3802    0.3800
    0.4102    0.4100
    0.4402    0.4400
    0.4602    0.4600
    0.4902    0.4900
    0.5002    0.5000
    0.5102    0.5100
    0.5302    0.5300
    0.5402    0.5400
    0.5602    0.5600
    0.5902    0.5900
    0.6102    0.6100
    0.6402    0.6400
```

Figure 5: Sniffer timestamps before and after time alignment

We observe that this approach yields *partially successful alignment* between the sniffer metadata and the raw RF data. In many cases, the extracted packets exhibit clear GFSK structure in the constellation plots, indicating that the packet boundaries are sufficiently accurate for demodulation and downstream processing. However, a subset of packets appear as noise, suggesting misalignment at the sample level.

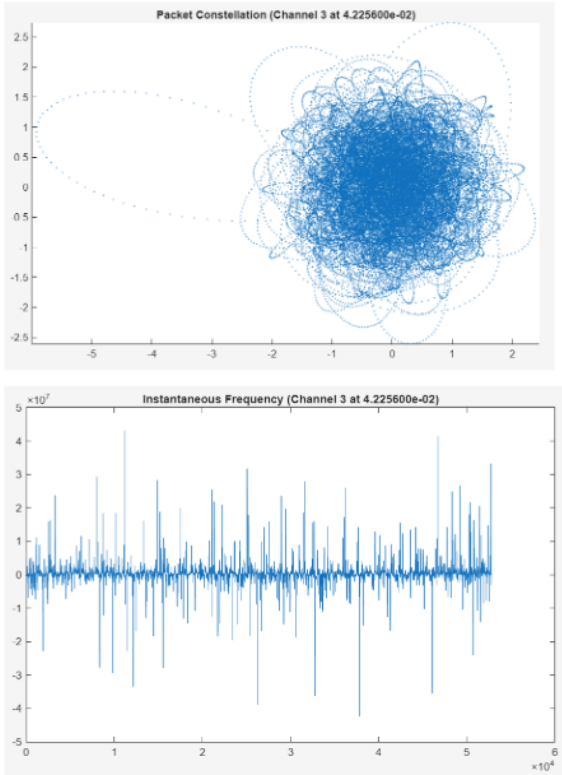


Figure 6: Packet Detection Using Sniffer Time Before for Channel 3

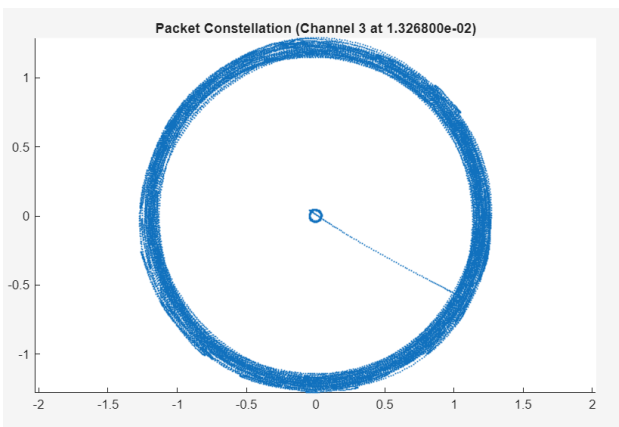


Figure 7: Packet Detection Using Time Synchronized Sniffer Time for Channel 3

Quantitatively, we observe the following approximate success rates across the three BLE advertising channels:

- Channel 37: ∼40–50% correctly aligned packets
- Channel 38: ∼70% correctly aligned packets
- Channel 39: ∼40–50% correctly aligned packets

Among the three channels, Channel 38 consistently provides the best performance, while Channel 37

exhibits the lowest reliability. This variation is likely due to differences in interference, channel occupancy, and propagation effects in the 2.4 GHz ISM band.

3.4 Impact of Time Synchronization

While this synchronization step ensures that timestamps are globally consistent and correctly spaced, we observe that it does not significantly improve packet extraction performance. In some cases, it introduces small deviations that slightly reduce alignment accuracy. This suggests that the dominant source of error is not global clock drift, but rather per-packet timing uncertainty.

Specifically, sniffer timestamps still do not correspond exactly to the start of the BLE packet preamble, but instead reflect detection events within the sniffer hardware. As a result, even with accurate global timing, individual packet boundaries may be offset by several samples, leading to occasional misalignment and noisy constellations.

3.5 Discussion and Limitations

The results indicate that sniffer timestamps provide a useful but imperfect proxy for packet timing in raw IQ data. While they enable coarse alignment and metadata association, they are not sufficiently precise for consistent sample-level packet extraction

In an earlier iteration of our SISO case, we implemented an alternative packet detection approach based on the received signal power, which yielded reasonable detection performance independent of sniffer timestamps.

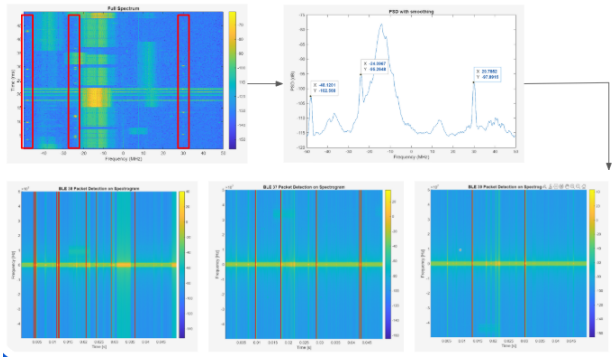


Figure 8: SISO Power-Based Packet Detection

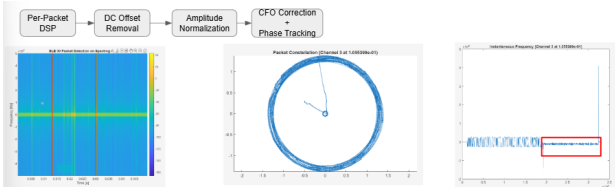


Figure 9: Per-Packet DSP Results for Alternative Packet Detection

This RF-based detection method could be leveraged to help align the sniffer timestamps by comparing them with the power-based packet detections, potentially enabling more robust alignment and improving overall packet extraction accuracy.

3.6 Extension to SIMO

While the SISO results demonstrate partial success in packet extraction using sniffer timestamps, extending this approach to a multi-antenna (SIMO) system provides a natural path for improvement. In our current

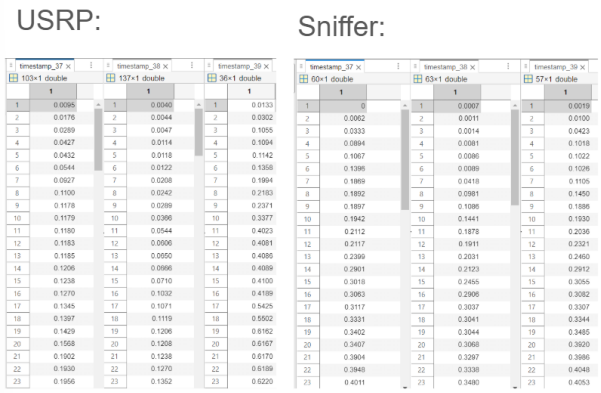


Figure 10: Misaligned Timestamps for Alternative Packet Detection

setup, we do not yet have the corresponding injected sync sniffer data for the SIMO capture. However, such data can be generated in the same manner as in the SISO case, using periodic BLE packet injection and sniffer-based timestamping.

Once properly synchronized sniffer data is available for the SIMO capture, the same time alignment and packet extraction pipeline can be applied. The key difference lies in the availability of two spatially separated observations of each transmission. For each detected packet, we independently extract the corresponding samples from both antennas and estimate their respective signal quality (SNR).

We then perform packet-by-packet antenna selection, choosing the antenna branch that provides the stronger and more reliable observation. This approach leverages spatial diversity to improve packet recovery performance. In particular, for packets that are already correctly aligned using sniffer timestamps, the SIMO system is expected to yield stronger and cleaner constellations compared to the SISO case, as at least one antenna is likely to observe the signal under more favorable channel conditions.

Overall, although full SIMO synchronization has not yet been implemented, the framework established in the SISO case directly extends to the multi-antenna setting. With properly aligned sniffer timestamps, the SIMO approach is expected to outperform SISO by exploiting spatial diversity and per-packet antenna selection.

4 Future Work

One of the original goals of our work was to use our time-aligned, fully-labeled IQ datasets to train ML models in Python for tasks such as device recognition, energy detection, and anomaly detection. However, partially due to our inability to decode a wide swath of the BLE spectrum because of sniffer limitations, this work falls outside of the scope of what can be achieved with our current toolset.

References

[1] T. Wang, C. -K. Wen, H. Wang, F. Gao, T. Jiang and S. Jin, “Deep learning for wireless physical layer: Opportunities and challenges,” in China Communications, vol. 14, no. 11, pp. 92-111, Nov. 2017, doi: 10.1109/CC.2017.8233654.

[2] K. Merchant, S. Revay, G. Stantchev and B. Nousain, “Deep Learning for RF Device Fingerprinting in Cognitive Communication Networks,” in IEEE Journal of Selected Topics in Signal Processing, vol. 12, no. 1, pp. 160-167, Feb. 2018, doi: 10.1109/JSTSP.2018.2796446.

[3] H. Jafari, O. Omotere, D. Adesina, H. -H. Wu and L. Qian, “IoT Devices Fingerprinting Using Deep Learning,” MILCOM 2018 - 2018 IEEE Military Communications Conference (MILCOM), Los Angeles, CA, USA, 2018, pp. 1-9, doi: 10.1109/MILCOM.2018.8599826.

[4] E. Fishler, A. Haimovich, R. S. Blum, L. J. Cimini, D. Chizhik and R. A. Valenzuela, "Spatial Diversity in Radars—Models and Detection Performance," in IEEE Transactions on Signal Processing, vol. 54, no. 3, pp. 823-838, March 2006, doi: 10.1109/TSP.2005.862813.

[5] Schellmann, Jungnickel and von Helmolt, "On the value of spatial diversity for the synchronisation in MIMO-OFDM systems," 2005 IEEE 16th International Symposium on Personal, Indoor and Mobile Radio Communications, Berlin, Germany, 2005, pp. 201-205, doi: 10.1109/PIMRC.2005.1651427.